

Ajay Kumar

PES1PG24CA128

 Quick Submit Quick Submit LIBRARY

Document Details

Submission ID**trn:oid:::1:3548740559****Submission Date****Apr 24, 2026, 11:54 AM GMT+5:30****Download Date****Apr 24, 2026, 11:58 AM GMT+5:30****File Name****PES1PG24CA128_AjayKumar_plag.pdf****File Size****2.8 MB****35 Pages****5,861 Words****35,634 Characters**

33% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



37 AI-generated only 31%

Likely AI-generated text from a large-language model.



3 AI-generated text that was AI-paraphrased 2%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



ABSTRACT

ShieldPatch is an intelligent cybersecurity platform designed for automated vulnerability detection and patch management. The primary objective of the system is to help users and organizations quickly identify software and file vulnerabilities and minimize the risk of exploitation. The platform integrates live threat intelligence feeds and machine learning techniques to assess vulnerability risk and recommend or deploy appropriate patches. ShieldPatch provides an easy-to-use dashboard that displays real-time system status, vulnerability analysis results, and patch guidance. By reducing the gap between vulnerability discovery and remediation, the system aims to enhance overall security and ensure safer computing environments with minimal manual effort.

1. INTRODUCTION

1.1 Introduction

As advancements are being made to digital technology through software applications at an explosive rate, there are also many more security problems associated with those software applications, standalone files, and the computer systems on which they run. Any of these items may retain vulnerabilities that hackers can use to attack the integrity, confidentiality, or availability of a computer system. Therefore, new cyber security threats require automated solutions that can rapidly identify all existing and new vulnerabilities while minimizing potential security risks.

ShieldPatch has developed an intelligent cyber security platform that automates the entire process of detecting vulnerabilities and providing patch management. To accomplish this, ShieldPatch inspects every component of a software application and the files associated with it in order to detect and identify regarding any existing or new share weaknesses. In addition, it leverages intelligent sources and analytic techniques to identify weaknesses for the user through a central dashboard with actionable data that the user can then use to make faster and better-informed security decisions.

1.2 Problem Definition

When businesses or individuals Delay on or fail to appropriately patch their systems, they can find it hard to keep secure. Many patches go untouched because of various reasons including not knowing when new updates are available, the complexity of the patching process, or lacking enough automated tools to help properly manage these patches. Because of this, the system is left open to attack from known vulnerabilities, thus leading to an increased chance of being infected by malware, losing data, or giving hackers a way in.

In addition to this there is an issue of effective Cybersecurity Management Gaps, because only a few solutions exist that integrate vulnerability detection; risk assessment, and patch recommendations. Therefore There is a need for an integrated solution that can automatically search out vulnerabilities, and offer assistance to users on what actions they need to take in order to mitigate these issues within their own organizations.

1.3 Proposed Solution

The ShieldPatch Program is a smart and automated response to identify potential security gaps (vulnerabilities) as well as to manage the process of updating the various software programs and files owned by an organization. This automated system uses threat intelligence (data) analytical methods to perform a continual review of software programs and files to identify any possible security vulnerabilities. Once a potential vulnerability is identified, ShieldPatch utilizes the level of severity of the vulnerability along with its associated risk profile to determine whether or not to issue a recommendation for a patch and the amount of time it would take to implement the patch.

In addition to automating the process of determining whether or not a vulnerability exists, ShieldPatch automates the entire process from determining the existence of a vulnerability to implementing patches for that vulnerability. By automating many of the manual steps involved in both the assessment and response processes, the amount of time it takes for an organization to respond to a potential security threat is reduced significantly. The ShieldPatch platform's intuitive interface enables users with varying levels of cybersecurity experience to view all details related to vulnerabilities.

1.4 Purpose of the Project

The focus of ShieldPatch is to decrease the time between vulnerability detection and mitigation by making it easier to find early warning signs of security risks and fixing them before an attacker takes advantage of them. One of the ShieldPatch project goals will provide straightforward instructions on what actions should be taken for discovered patches; thus, simplifying the vulnerability management process. Ultimately, the result will be greater system security, a decrease in the risks associated with operating the system, and the fostering of proactive behaviours associated with protecting against cyber threats.

1.5 Scope of the Project

A project called ShieldPatch will be creating vulnerability scans on both software, and also files to provide risk assessments of detected vulnerabilities and suggestions on how to patch those vulnerabilities from the threats posed by the presence of those vulnerabilities via their threat intelligence. By utilizing ShieldPatch, users will be able to make more informed and timely decisions in order to better secure their systems.

The purpose of the ShieldPatch project is to provide the individual user and enterprise with an avenue to improve their defenses against cyber-attacks; however, the functionality only provides for vulnerability discovery and patching suggestions but does not provide for automation of patching or for real-time intrusion detection.

2. LITERATURE SURVEY

2.1 Related Work

Paper 1:

Title: Cyber Threat Intelligence Mining for Proactive Cybersecurity Defense: A Survey

Authors: Nan Sun, Junhua Yan, Weijie Xie, Rui Zhang

Publication Details: IEEE Access, Vol. 11, 2023

Summary: This work is a thorough review of CTI mining techniques applied towards proactive cybersecurity defenses. Sources of data, as well as machine learning (ML) methods and analytical methodologies for collecting and correlating threat information are discussed. Furthermore, the authors demonstrate how real time CTI can be used to predict future cyber threats and assist organizations in making security decisions; however, this work consists predominantly of theoretical analysis and does not provide any automated means to remediate vulnerabilities or manage patching.

Paper 2:

Title: Exploit Prediction Scoring System (EPSS)

Authors: Jay Jacobs, Sasha Romanosky, Idris Adjerid, Michael Baker

Publication Details: FIRST.org, EPSS v3, 2023

Summary: The Exploit Prediction Scoring System (EPSS) is introduced in this article as a predictive framework based on data that estimates how vulnerable systems will be exploited in the real world. Using historical data, EPSS applies machine learning algorithms to predict the exploitability of known vulnerabilities and then assists users in prioritizing remediation efforts by determining the likelihood of being exploited.

While EPSS can help organizations prioritize their vulnerabilities based on risk factors, it does not provide any direction regarding how to implement mitigations for those vulnerabilities or automatically deploy patches.

Paper 3:

Title: Improving Exploit Prediction Using Word Embedding Techniques

Authors: Xin Zou, Weilin Xu, Zhiqiang Lin

Publication Details: Proceedings of the ACM Conference on Computer and Communications Security, 2022

Summary: The current study examines how different types of word embedding techniques can be utilized to provide a more accurate assessment of vulnerability exploitation likelihood through an analysis of the descriptive text of vulnerabilities. The proposed methodology intends to take advantage of the semantic relationship between words within the descriptive text to improve predictive performance. The proposed method is able to successfully identify vulnerabilities that pose a higher level of risk; however, it does not incorporate any method of providing recommendations for applying a patch or mitigation of the vulnerability.

Paper 4:

Title: Large Language Models for Automated Vulnerability Patch Prioritization

Authors: Rohan Patel, Ananya Mehta, Karthik Rao

Publication Details: arXiv Preprint, arXiv:2308.11245, 2023

Summary: In this study, large language models (LLMs) are used to automate the prioritization of vulnerability patches by evaluating vulnerability reports and patch notes to generate a list of prioritised patches based on both model and contextual relevance. This method shows good results for the classification of vulnerabilities; however, it is mostly reliant on text data without conducting any real-time scanning for vulnerabilities.

Paper 5:

Title: SmartPatch: An Intelligent Framework for Vulnerability Assessment and Patch Recommendation

Authors: Li Wang, Chen Yu, Hao Zhang

Publication Details: Journal of Cybersecurity and Information Systems, Vol. 9, No. 2, 2022

Summary: A smart patch proposes an intelligent framework that combines vulnerability scanning and patch recommendation techniques. The system evaluates vulnerabilities based upon established criteria and makes suggested patches based on those evaluations. While the framework improves the efficiency of vulnerability management, it does not include any form of adaptive intelligence. The current SmartPatch framework contains no provision for live threat intelligence and does not utilize any learning based risk assessment procedures.

Paper 6:

Title: VALIDATE: A Dataset and Framework for Vulnerability Assessment

Authors: Adam Turner, Michael Smith, Laura Chen

Publication Details: Proceedings of the IEEE International Conference on Cybersecurity, 2022

Summary: VALIDATE is a new dataset released for assessing models that conduct vulnerability assessments and created as structured data to allow for a systematic way to analyze vulnerabilities based on pre-defined standard metrics and using pre-determined benchmarks. The VALIDATE dataset will assist in furthering research into vulnerability assessments and risk evaluations, but this dataset does not include any features for automating patch recommendations or integrating with real-time threat intelligence agencies.

Paper 7:

Title: ILLATION: Intelligent Learning-Based Vulnerability Risk Prioritization

Authors: Ahmed Hassan, Priya Nair, Daniel Brown

Publication Details: Journal of Information Security and Applications, Vol. 67, 2022

Summary: ILLATION offers a new approach for sorting through vulnerabilities; it uses historical vulnerability data combined with machine-learning models to create risk-level rankings for vulnerabilities. Its approach will make it easier to prioritize vulnerabilities; however, this solution only focuses on scoring vulnerabilities by overall risk level (as opposed to actionable patch management or mitigation).

Paper 8:

Title: Exploit-Aware Vulnerability Remediation Using Machine Learning

Authors: Thomas Lee, Rebecca Wong, Samuel Kim

Publication Details: Computers & Security Journal, Vol. 115, 2022

Summary: The suggested method examines existing patch recommendation systems and identifies their limitations, concluding that most of these recommendations are either outdated or do not consider recent changes in vulnerability and threat landscape. It then creates an innovative framework to exceed current capabilities by incorporating exploit predictions into the vulnerability management workflow while continually updating the recommendations based on recent data, thereby allowing organizations to take corrective action against vulnerabilities within their software before they can be exploited by attackers.

Paper 9:**Title:** ExPreHet: A Graph-Based Approach for Exploit Prediction**Authors:** Jian Liu, Hao Wu, Ming Zhao**Publication Details:** IEEE Transactions on Dependable and Secure Computing, 2023

Summary: The ExPreHet model represents vulnerabilities, exploitation, and systems by constructing exploit prediction information using a graph-based modeling methodology that more accurately portrays the intricate relationships among those vulnerabilities, exploits, and system components. The utility of this approach is primarily realized via the enhancement of prediction effectiveness by incorporating dependencies of nodes using heterogeneous graphs. While the methodology for creating and/or maintaining this graph-based predictive model is effective for predicting exploits, it does not lend itself to vulnerability remediation or automated patch management.

Paper 10:**Title:** A Systematic Literature Review on Machine Learning-Based Vulnerability Detection**Authors:** Sofia Martinez, Luca Romano, Emily Johnson**Publication Details:** ACM Computing Surveys, Vol. 55, No. 4, 2023

Summary: The design of this systematic review is to conduct a comprehensive analysis of techniques and methodologies using machine learning for identifying vulnerabilities in software systems. This paper provides an overview of all identified methodologies (i.e., techniques), data sets, and metrics utilized in evaluating ML vulnerability detection techniques while pinpointing a gap in research related to automation and real-time vulnerability management solutions. The results from this paper emphasize the importance of developing and implementing systems that integrate the three components (i.e., detection, risk assessment, and remediation) into a single solution to enhance current capabilities for detecting vulnerabilities; this will support innovations like ShieldPatch.

2.2 Existing Systems

Traditional CVSS-Based Vulnerability Scanners:

The Common Vulnerability Scoring System (CVSS) is used by the majority of vulnerability scanners to assign severity levels for all software vulnerabilities. When a vulnerability is identified, the scanner compares the configuration of the system with a database of known vulnerabilities, where each vulnerability is assigned a risk score based upon the CVSS guidelines. All CVSS-compliant scanners use the same methodology to evaluate vulnerabilities and output a vulnerability assessment. However, the CVSS standard has not been frequently updated to reflect evolving security threats, and therefore scanners using this system do not incorporate any real-time information relating to the ongoing exploitation of the vulnerability in question, nor do they typically consider the environmental context in which the vulnerability exists. Consequently, very high-rated vulnerability may not pose an immediate threat to your network's security, whilst low-rated vulnerabilities currently being exploited by attackers may go undetected.

Exploit Prediction Scoring System (EPSS):

EPSS, or the Exploit Prediction Scoring System, is intended to generate metrics based on real-world data to evaluate the likelihood that there will be an exploit for each vulnerability existing in production systems. For that reason, the EPSS leverages historical data regarding attempted exploits as well as statistical models driven by both it and the previous tried exploits to create the equations the EPSS uses to form the metrics that rank vulnerabilities according to their likelihood of being exploited. As a result of the method that EPSS uses to create the basis for ranking vulnerabilities and how they will likely be exploited — this approach to prioritizing the vulnerabilities is based on total risk, which is substantially superior to any traditional vulnerability scoring system (that relies primarily on static information) — security staff have to conduct manually the assessment for those vulnerabilities that were found to be exploitable, so that the appropriate steps can be taken to remediate (e.g., create an appropriate patch).

SmartPatch Framework:

SmartPatch is a framework that helps organizations manage vulnerabilities and deploy patches efficiently by providing an approach for defining the best order to deploy those patches within a complex system such as an industrial control or Internet of Things (IoT) environment. The objective of this framework is to reduce system downtime and mitigate patch failures when deploying patches. While SmartPatch provides better efficiency in planning for patching, it has several limitations. SmartPatch can only be used for particular systems and resources and has limited flexibility for rapidly adapting to new operating systems and application environments. SmartPatch also does not utilize live threat intelligence or learning-based risk assessment.

Machine Learning-Based Vulnerability Prioritization Systems:

A number of machine learning-based systems have been proposed for better prioritizing vulnerabilities including ILLATION and text-based exploit prediction models. Different tools analyze prior vulnerability and exploit datasets they are able to make accurate predictions on vulnerabilities. These machine learning-based systems will learn to identify high-risk vulnerabilities based on complex patterns identified through their learned processes. Unfortunately most of the existing models have only been developed for research purposes and operate in offline environments with no real-time threat intelligence or automated patch recommendations.

Commercial Vulnerability Management Tools:

Vulnerability management tools that you can purchase will centralize dashboards for scanning, reporting, and compliance monitoring of your enterprise's systems. The tools provide a wide array of visibility into your systems' vulnerabilities; but, generally, they require a lot of manual labor for assessment and remediation. In addition, because these tools are proprietary, there is very little transparency associated with them, which can result in users being unsure about how to prioritize vulnerabilities or choose the appropriate patch, ultimately limiting their ability to use the tools for proactive management of security.

Features	CVSS- Based Scanners	EPSS	SmartPatch	ML-Based Systems	Commercial Tools	ShieldPatch
Real-Time Vulnerability Detection	No	No	No	Partial	Yes	Yes
Exploit Likelihood Prediction	No	Yes	No	Yes	Partial	Yes
Use of Live Threat Intelligence	No	Partial	No	No	Partial	Yes
Adaptive / Learning- Based Analysis	No	No	No	Yes	No	Yes
Automated Patch Recommendation	No	No	Partial	No	Partial	Yes
Explainable & Transparent Dashboard	No	No	No	No	Partial	Yes

3.1 HARDWARE REQUIREMENTS

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent (or higher)
System Architecture	64-bit
Main Memory (RAM)	Minimum 8 GB
Storage	Minimum 50 GB free disk space
Display	Resolution of 1366 × 768 or higher
Network Interface	Wired or wireless network support
Internet Connectivity	Required for vulnerability lookup, threat intelligence feeds, and patch updates

3.2 SOFTWARE REQUIREMENTS

Category	Tools / Frameworks	Version
Operating System	Windows / Linux / macOS (64-bit)	Windows 10+, Ubuntu 20.04+, macOS 11+
Programming Language	Python	3.10+
Backend Framework	Flask (RESTful API)	2.3+
Frontend Technologies	React.js, HTML5, CSS3, Bootstrap	React 18+, Bootstrap 5
Database	MySQL	8.0+
Machine Learning Libraries	Scikit-learn (TF-IDF Vectorizer, Random Forest Regressor, Pipeline)	1.1.3
Threat Intelligence & Data Fetching	CVE Feeds, EPSS, ExploitDB, Requests, BeautifulSoup	2.32.5 , 4.14.3
System Scanning	OSQuery	5.19.0
Containerization & Sandbox	Docker	24+
Development & Version Control	Git, VS Code, PyCharm, WebStorm	1.107.1
API Testing & Debugging	Postman	11.77.1
Operating Environment	Python Virtual Environment (venv)	3.10.19

4. SOFTWARE REQUIREMENTS SPECIFICATION

4.1 Users

ShieldPatch supports many types of users based on their requirements for monitoring system health and cybersecurity. System Administrators are the primary users who configure vulnerability scans, manage system policies, monitor detected vulnerabilities and apply patch recommendations from the centralized dashboard.

Security Analysts use the system to analyze trends in vulnerability, assess risk level, and use real-time threat intelligence to identify and mitigate potential threats. Normal users such as individual system owners and employees utilize ShieldPatch to scan their systems and check overall health status, vulnerability status and recommended security actions.

ShieldPatch is designed for ease of use by both technical and non-technical users so they can utilize the system effectively.

4.2 Functional Requirements

User Authentication and Access Control:

The ShieldPatch system includes secure authentication mechanisms that limit administrator access/ability to manage system operations. Therefore, operations related to vulnerability scanning, patch management reporting, and system configuration features will only be available to authorized personnel.

File and System Scanning:

Using OSQuery and file analysis software, administrators are able to run scans against all uploaded files (APKs, EXEs) and components on the system. This feature helps to identify any software vulnerabilities, detect any malicious behaviours, and identify security threats on the system.

Threat Intelligence Integration:

Data is collected and constantly updated about live vulnerabilities and exploits occurring in the environment by using trusted Threat Intelligence sources including: CVE Repositories, the National Vulnerability Database (NVD), Exploit Prediction Scoring System (EPSS) and Exploit Database (ExploitDB). By doing this, Vulnerability Detection and Assessment is kept accurate and current.

Machine Learning–Based Risk Prediction:

Risk scores for vulnerabilities that have been identified are calculated using a machine-learning model. Risk scores give an indication of the likelihood that a certain vulnerability will be exploited in relation to its severity and real-world context.

Patch Recommendation:

The system generates verified and compatible patch recommendations for identified vulnerabilities. This assists administrators in selecting appropriate remediation actions to reduce security risks.

Sandbox Testing Environment:

Through utilizing virtualization technologies, the system creates a separate sandbox environment that allows for safe testing of recommended patches before they are deployed. This minimizes the likelihood of instability or system failure due to poorly written patches.

Rollback Mechanism:

If a patch installation does not succeed or causes unanticipated anomalies, the system will automatically return to the previous working state, giving them stability, assurance, and peace of mind that their data and their system will remain up and functioning properly no matter what happens after they have made changes to their computer.

AI Chatbot Assistance:

An artificial intelligence-driven chatbot provides support for the administrator by explaining vulnerabilities that have been detected, providing patching instructions, and answering questions related to security. This eliminates a lot of manual activity and enables more efficient decision-making.

Dashboard and Reporting:

The platform provides a live overview of the vulnerability characteristics of a system as well as the risk associated with it, patch statuses, and how healthy the system is at any moment. Furthermore, the platform compiles thorough reports to allow for tracking, auditing, and documentation.

Alerts and Notifications:

The system sends instant alerts and notifications to administrators for newly detected vulnerabilities, high-risk threats, failed patches, or critical system updates. This ensures timely response to security incidents.

4.3 Non-Functional Requirements

Performance:

The vulnerability scanning and analysis process will take less than a reasonable amount of time. The system was designed to accommodate the rapid processing of system and file data and allows users to receive scan results and risk assessments in an efficient manner.

Security:

The system provides secure handling of vulnerability data, system configuration, and user information. Access to administrative features is restricted through authentication mechanisms. Sensitive data is secure from unauthorized access.

Reliability:

The purpose of the system is to function correctly in a normal environment and features tools like rollback support that help ensure continued system stability. Should there be an error when applying a patch, the system will return back to a stable state after recovery.

Usability:

A user-friendly interface is part of the system. Technical and non-technical both can "get" engaged with ShieldPatch through the interface. Users get a clear and easy to understand view on the dashboard and report on their vulnerability status and what actions they are suggested to take to address their identified vulnerabilities.

Maintainability:

Maintenance and Upgrades Are Easy to Do in This Configuration of Systems. The System Is Made Up of Modules and Uses Standardized Frameworks, Therefore Modifications/Enhancements Can Occur with Little or No Effect on Existing Functionalities.

Portability:

ShieldPatch is platform-independent and can operate on different operating systems such as Windows, Linux, and macOS. This allows flexibility in deployment across diverse computing environments.

5. SYSTEM DESIGN

5.1 Architecture Diagram

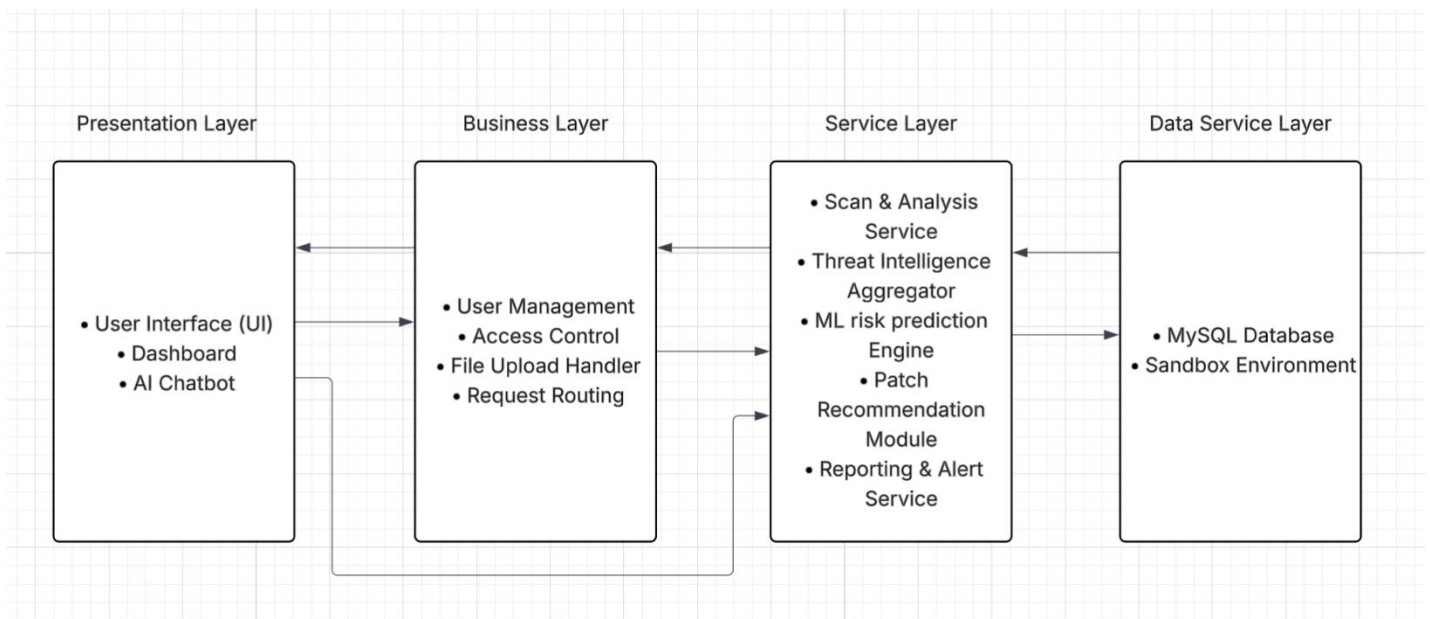


Fig. 5.1: Architecture Diagram

The architecture of ShieldPatch system is represented in this diagram. The four layers of the system - Presentation Layer, Business Logic Layer, Service Layer and Data Service Layer - each have their own specific functions: user input, processing user's request, analysing vulnerabilities and storing data.

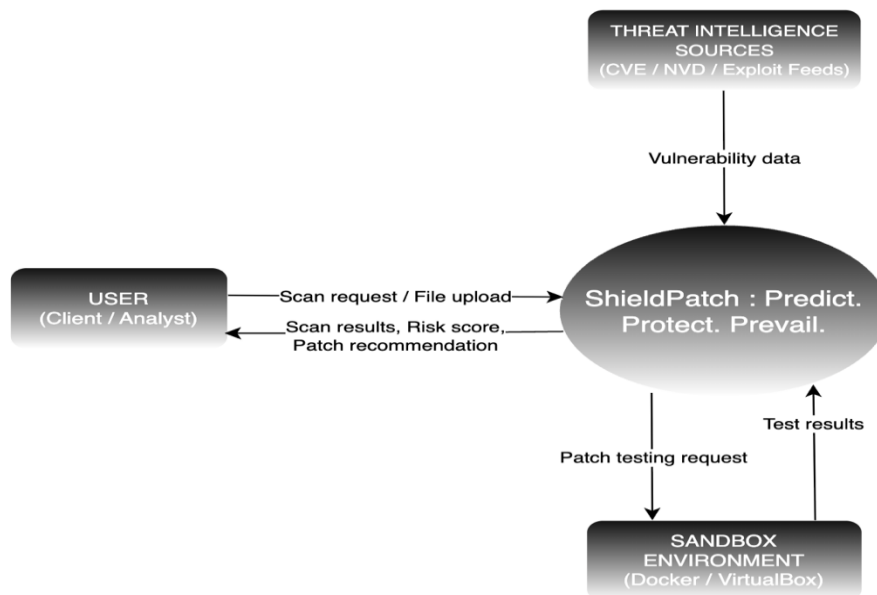


Fig. 5.2: Context Flow Diagram

The diagram displays the processes that occur between a user and the ShieldPatch system as well as with external sources for Threat Intelligence and the Sandbox environment. It illustrates how test scan requests are submitted to the ShieldPatch system, processes carried out in the ShieldPatch system, and then the delivery of test scan results, risk scores, and patch recommendations to the user.

6. DETAILED DESIGN

6.1 Process Flow Diagram

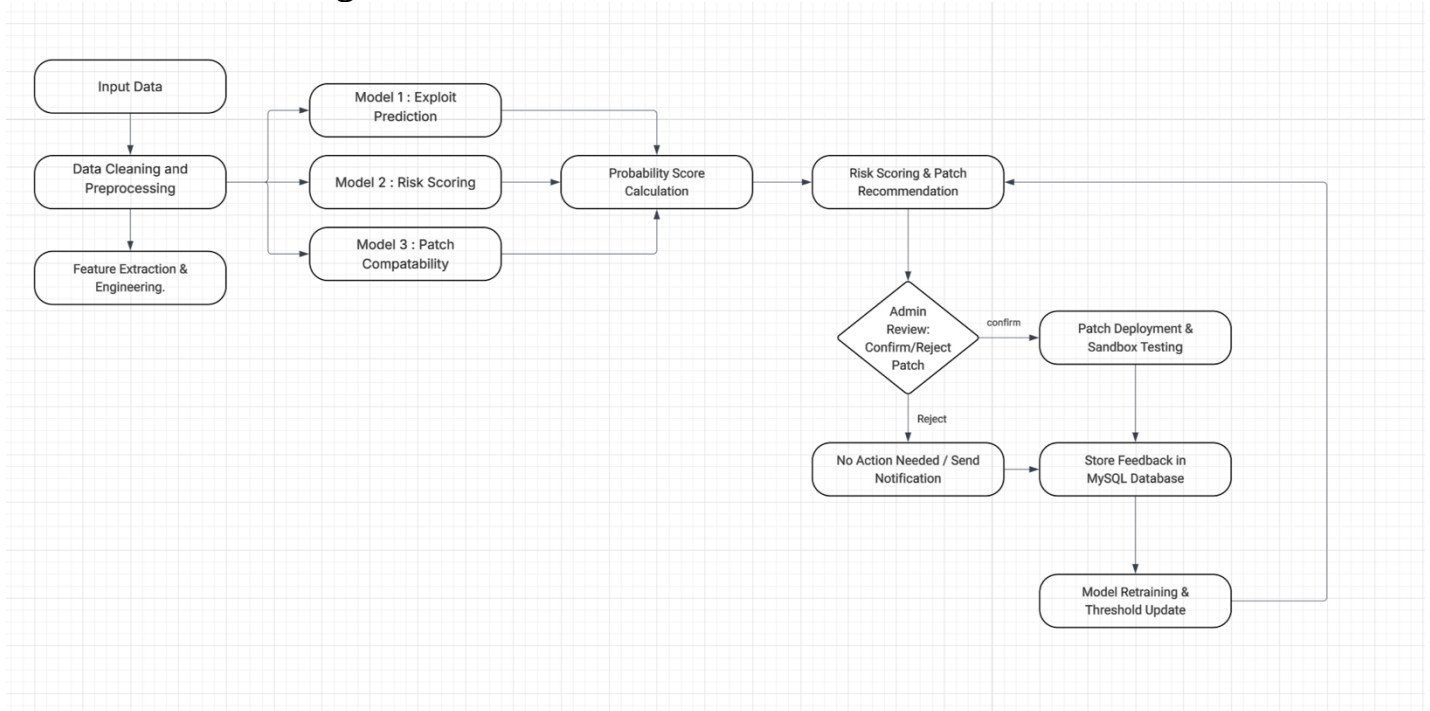


Fig. 6.1: Process Flow Diagram

This image demonstrates how the entire workflow of ShieldPatch works. It includes the steps needed to get an input file cleaned; have it passed through many machine learning models to predict exploits; generate risk scores; check patch compatibility; make patch recommendations; test the patch in a sandbox; and finally provide a place to store feedback for future retraining of the models.

6.2 Use Case Diagram

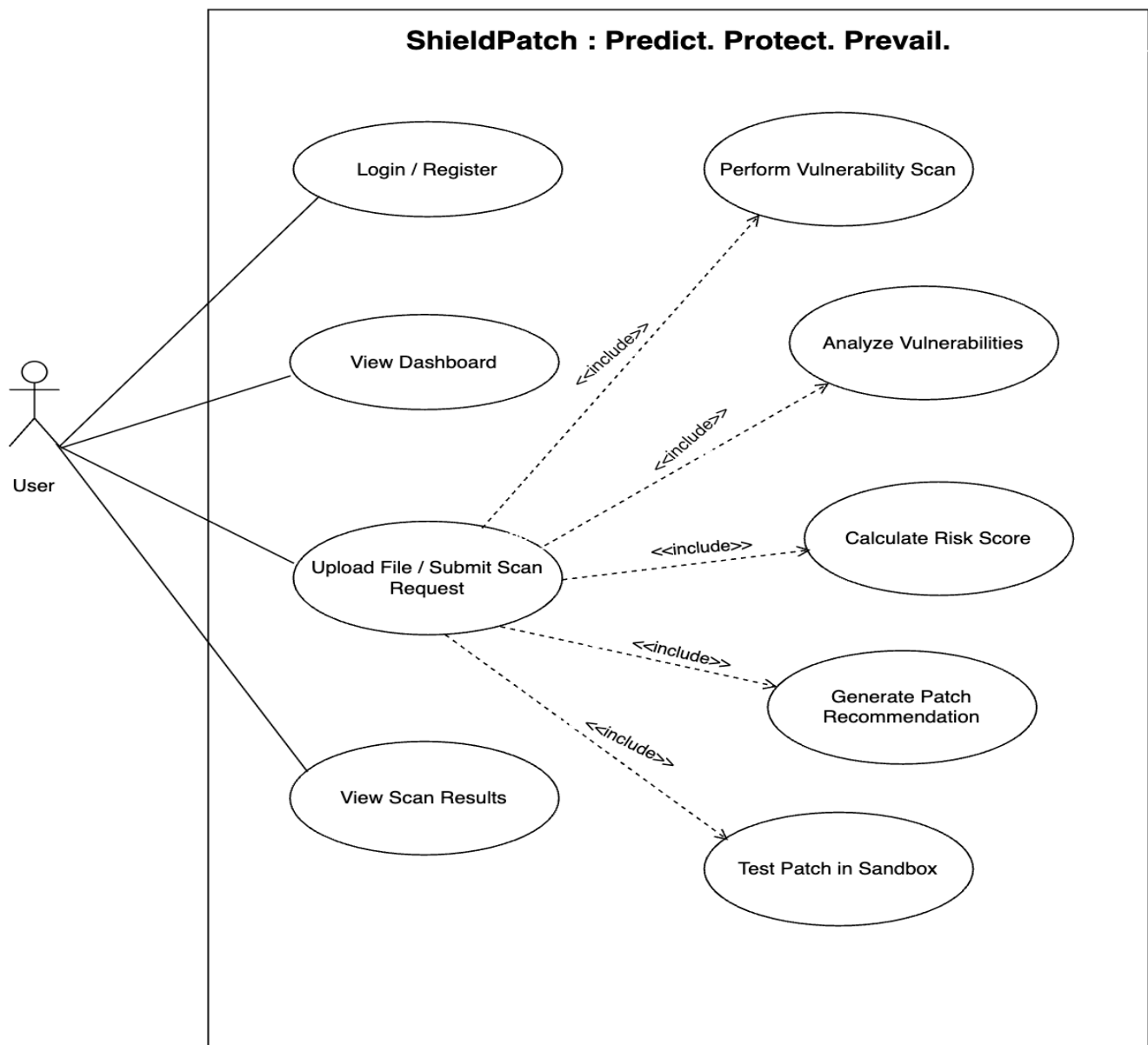


Fig. 6.2: Use Case Diagram

This diagram illustrates how a user interacts with the ShieldPatch system. It contains the core features of how to sign in, access your dashboard, create a scan request, conduct a vulnerability scan, obtain a risk score, receive patch recommendations, test a patch in a sandbox, and view results from a scan.

6.3 Sequence Diagram

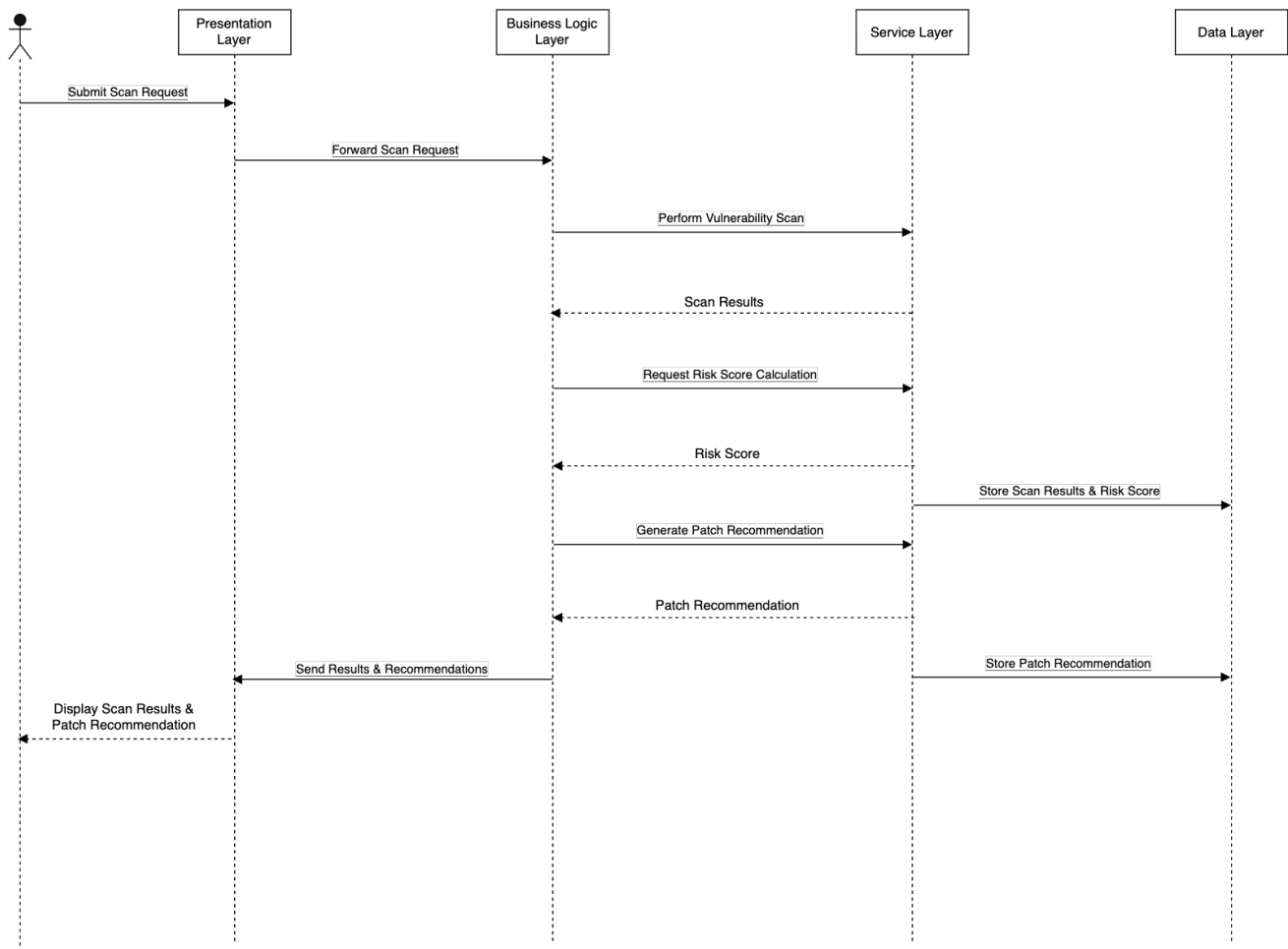


Fig. 6.3: Sequence Diagram

This diagram outlines the different layers of interaction from the user to the presentation layer, business logic layer, service layer, and data layer. Furthermore, this diagram describes how a scan request moves through each of the system components, how vulnerability analysis and risk scoring occurs, and how to return results and patch recommendations will be accomplished and stored.

6.4 Database Schema Diagram

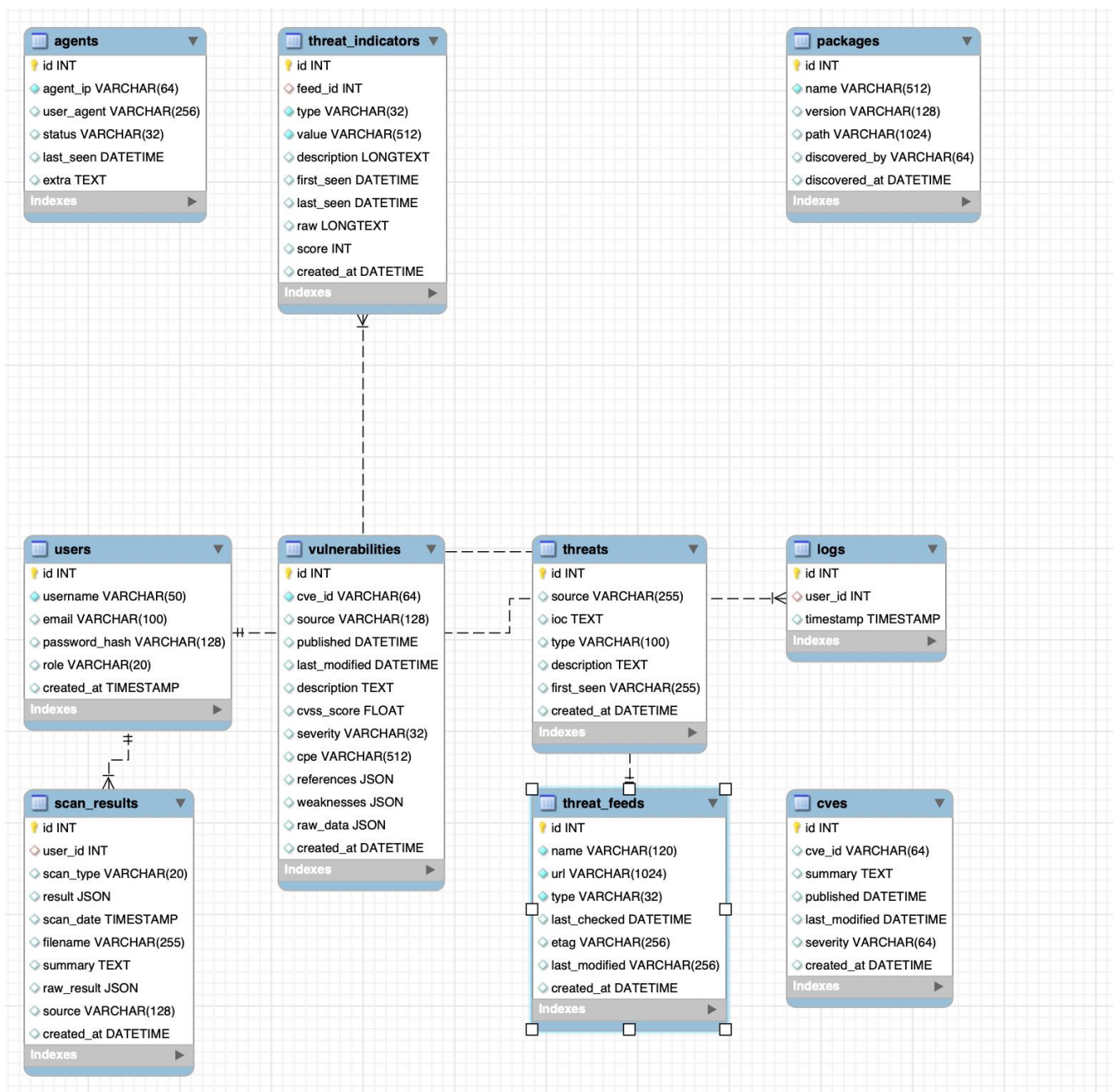


Fig. 6.4: Database Schema Diagram

A graphical representation of the ShieldPatch database's architecture that consists of tables (users, scan results, vulnerabilities, threats, threat feeds, logs, and packages), as well as their corresponding attributes and relationships which allow for effective storage/retrieving of security data.

6.5 Entity Relationship (ER) Diagram

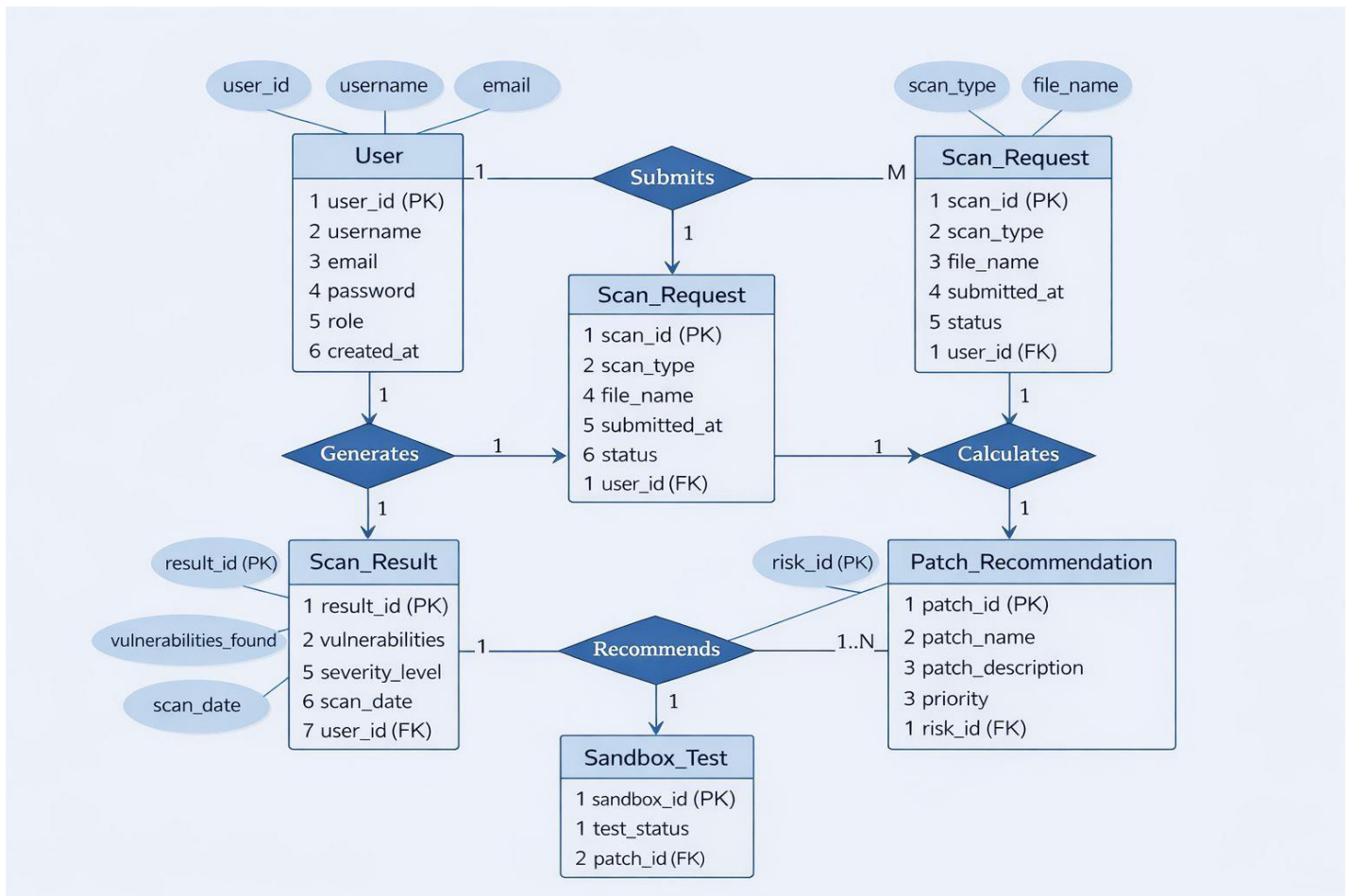


Fig. 6.5: Entity Relationship (ER) Diagram

The diagram indicates the logic behind relationships to the entities within the ShieldPatch system. Users will request scans, scan, find their risk score, receive recommendations, and sandbox testing on patches with primary and foreign keys represented as well.

7.1 Pseudo Code with Description

Data Acquisition & System Scanning

BEGIN Data_Acquisition

SCAN installed system software

SCAN standalone files (APK, EXE)

FETCH threat intelligence from:

CVE feed

EPSS feed

ExploitDB feed

COMBINE scanned system data with threat intelligence

END Data_Acquisition

Description: This phase collects system data and gathers vulnerability information from external threat intelligence sources.

Preprocessing Phase

BEGIN Preprocessing

CLEAN scanned system data

REMOVE duplicate and irrelevant entries

PERFORM deep file analysis using:

OSQuery

Androguard

pefile

PowerShell / Bash scripts

AGGREGATE analyzed results with threat intelligence

NORMALIZE all data formats

PREPARE dataset for machine learning based scoring

END Preprocessing

Model Phase (Machine Learning Risk Prediction)

BEGIN ML_RiskPrediction

LOAD trained machine learning pipeline:

TF-IDF Vectorizer

Random Forest Regressor

FOR each detected vulnerability

EXTRACT relevant textual and contextual features

TRANSFORM input data using TF-IDF Vectorizer

PREDICT risk and exploitability score using Random Forest Regressor

PRIORITIZE vulnerabilities based on predicted risk score

END FOR

END ML_RiskPrediction

Description: This phase applies machine learning techniques to predict vulnerability risk and prioritize threats.

Patch Recommendation and Testing

BEGIN PatchRecommendation

IDENTIFY high-risk vulnerabilities based on predicted risk scores

GENERATE recommended patches or mitigation fixes

FOR each recommended patch

SIMULATE patch deployment in a sandbox environment

(Docker or Virtual Machine)

IF patch simulation fails THEN

PERFORM rollback operation

MARK patch as unsafe

ELSE

MARK patch as approved for application

END IF

END PatchRecommendation

Description: This phase recommends patches and validates them in a secure sandbox environment before deployment.

AI Guidance and Dashboard Phase

BEGIN AIGuidance_Dashboard

UPDATE dashboard with:

- detected vulnerabilities
- associated risk levels
- patch recommendation status
- overall system health metrics

PROVIDE instant explanations using AI chatbot

(Rasa integrated with Gemini API)

DISPLAY security insights and guidance to administrator

END AIGuidance_Dashboard

Description: This phase presents vulnerability insights and recommendations through an interactive dashboard and AI chatbot.

Automated Action and Logging Phase

BEGIN Automated_Action_Logging

AUTOMATICALLY apply approved patches to target system

IF patch application causes system instability THEN

PERFORM automatic rollback to previous stable state

MARK patch as failed

ELSE

CONFIRM successful patch deployment

END IF

LOG system activities including:

- vulnerability scan results
- applied patches

system status updates

STORE all logs in the database for monitoring and audit purposes

END Automated_Action_Logging

Description: This phase handles patch application, rollback, and system activity logging for monitoring and auditing.

Reporting and Continuous Learning Phase

BEGIN Reporting_Learning

SEND automated alerts for:

critical vulnerabilities

failed patch applications

unusual system activity

GENERATE detailed system reports for:

vulnerability status

patch outcomes

rollback events

overall system health

COLLECT outcome data from:

vulnerability scans

patch application results

rollback logs

STORE collected data for analysis and model improvement

PERFORM periodic offline model retraining

using validated historical outcome data

UPDATE prediction model to improve future risk assessment accuracy

END Reporting_Learning

Description: This phase generates reports and continuously improves the system using historical data.

7.2 Screenshots (Implementation Output)

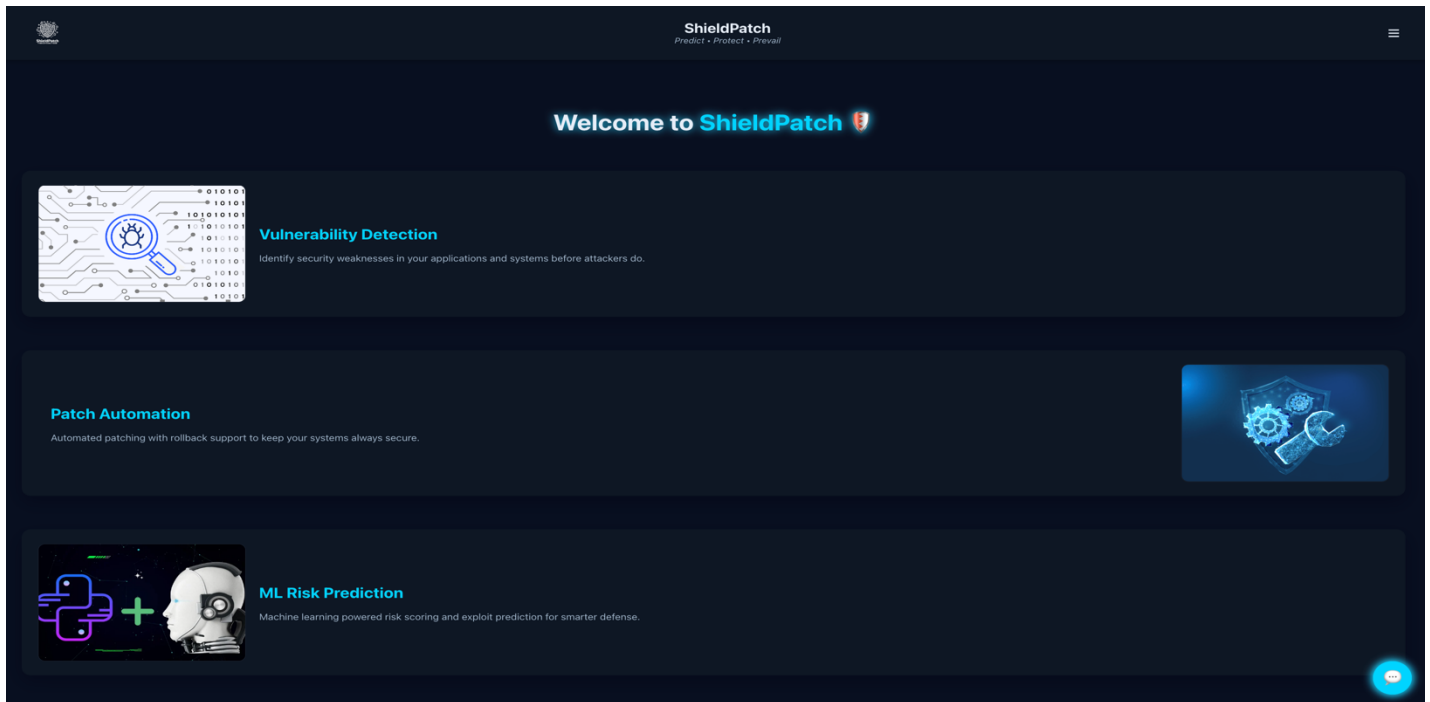


Fig. 7.2.1: ShieldPatch Home Page

This figure shows the home page of the ShieldPatch system. It introduces the core features such as vulnerability detection, patch automation, and machine learning-based risk prediction.

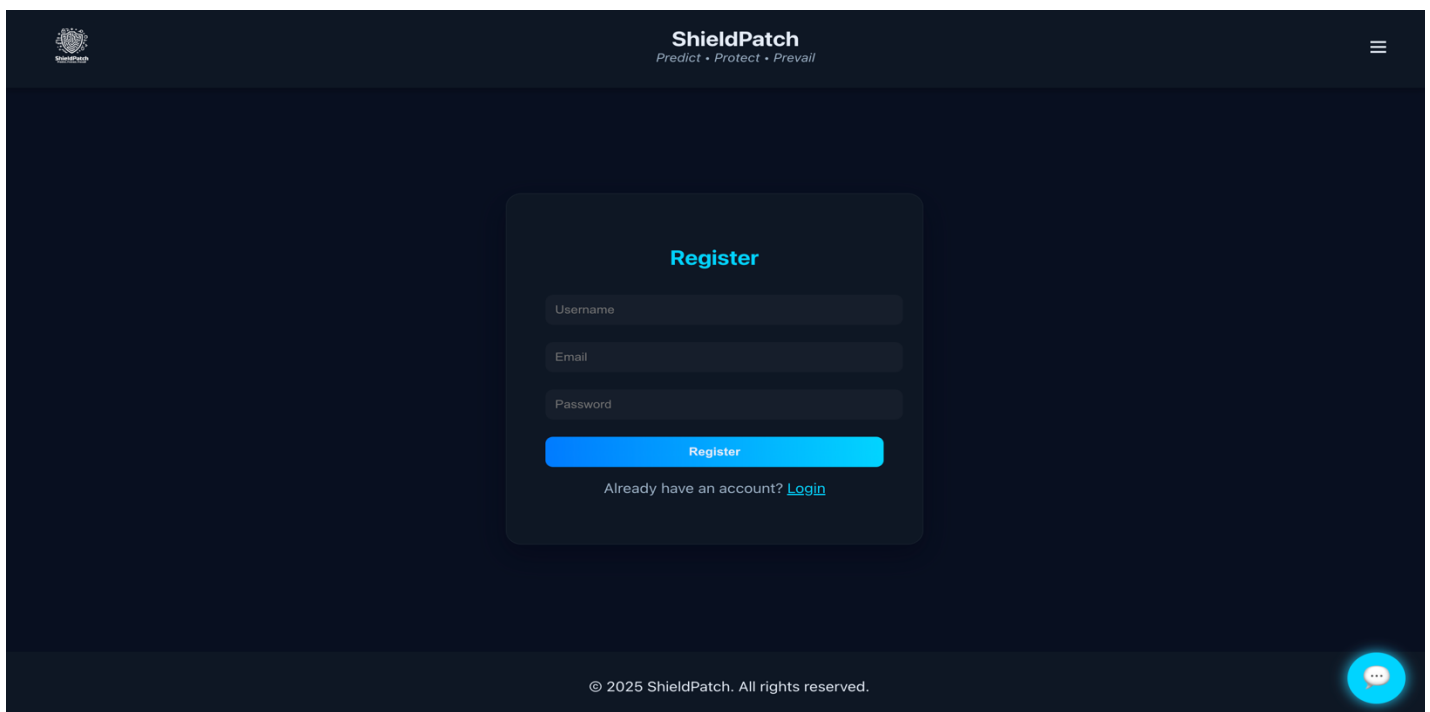


Fig. 7.2.2: User Registration Page

This figure illustrates the user registration interface, allowing new users to create an account to access ShieldPatch functionalities securely.

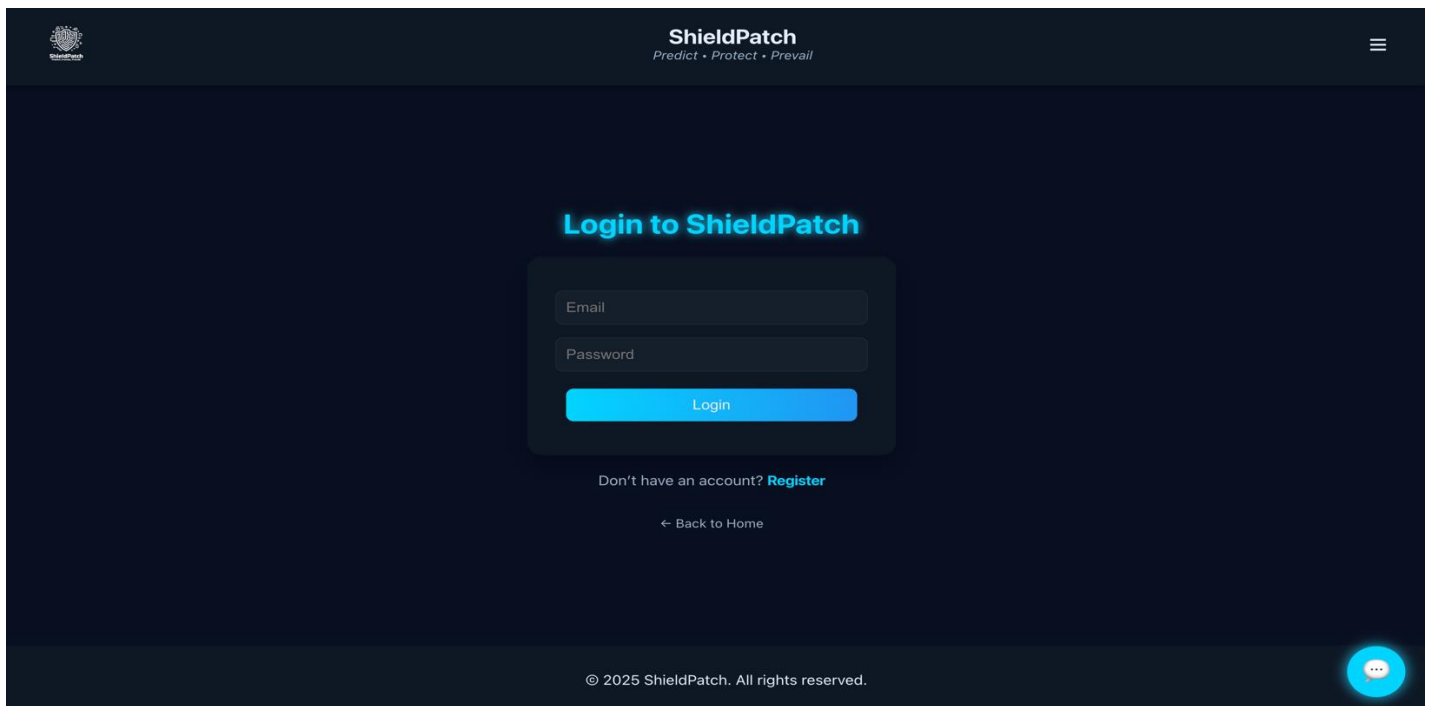


Fig. 7.2.3: User Login Page

This figure shows the login interface through which registered users authenticate themselves to access the ShieldPatch dashboard.

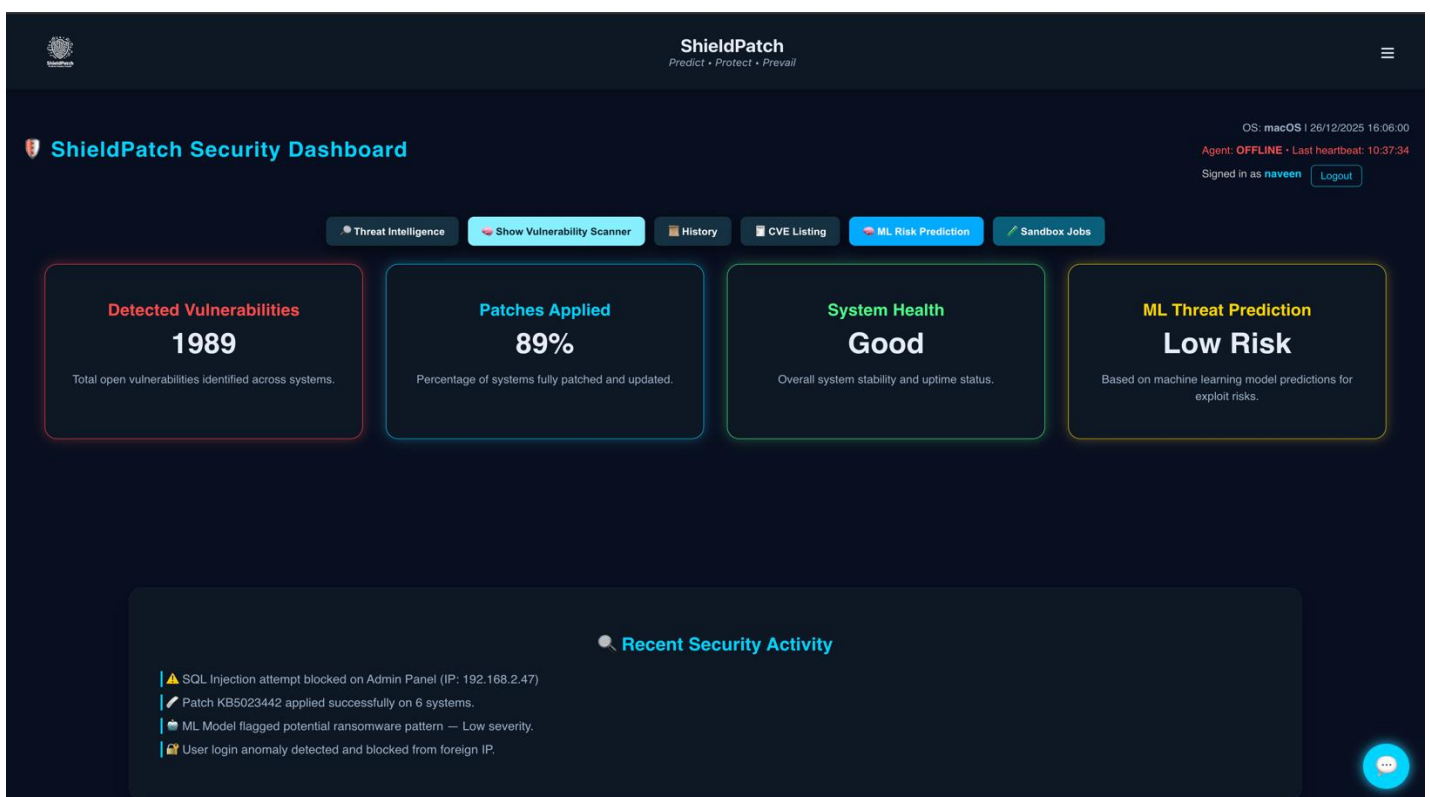


Fig. 7.2.4: ShieldPatch Security Dashboard

This figure presents the main security dashboard of ShieldPatch, displaying real-time vulnerability statistics, system health status, and machine learning-based threat predictions.



ShieldPatch
Predict • Protect • Prevail

ShieldPatch Vulnerability Scanner

Start System Scan

Scan complete! Results and discovered software updated.

Upload File for Scan

ID	SOFTWARE	CVE	DESCRIPTION	SEVERITY	RISK	ACTION
101	OpenSSL	CVE-2024-12345	Buffer overflow in TLS handshake	High	95	Patch Now
102	ExampleApp v1.2.3	CVE-2023-54321	Improper input validation	Critical	100	Patch Now
103	LocalDaemon		Weak permissions on config file	Medium	41	✓ Patched

Discovered Software

NAME	VERSION	PATH	VULNERABLE	LINKED CVE	RISK
Code	1.107.1	/Applications/Visual Studio Code.app	No	-	-
Telegram	12.3	/Applications/Telegram.app	No	-	-
Word	16.104	/Applications/Microsoft Word.app	No	-	-
Docker	4.54.0	/Applications/Docker.app	No	-	-
Safari	28.2	/System/Volumes/Preboot/Cryptexes/App/System/Applications/Safari.app	No	-	-
Excel	16.104	/Applications/Microsoft Excel.app	No	-	-
ChatGPT	1.2025.343	/Applications/ChatGPT.app	No	-	-
WhatsApp	25.37.76	/Applications/WhatsApp.app	No	-	-

Fig. 7.2.5: Vulnerability Scanner Output

This figure shows the vulnerability scanning module where detected software vulnerabilities are listed along with CVE details, severity levels, risk scores, and patch status.

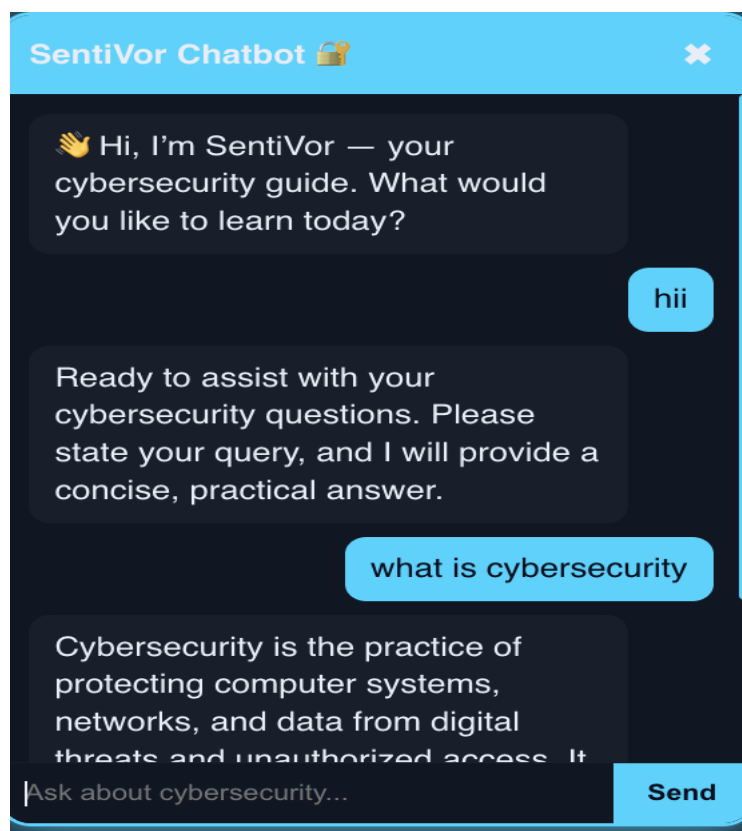


Fig. 7.2.6: AI Guidance Chatbot Interface

This figure illustrates the AI-powered guidance chatbot integrated into ShieldPatch, which assists users by answering cybersecurity-related queries and explaining detected vulnerabilities.

8. SOFTWARE TESTING

8.1 Model Testing

```
(venv310) ajaykumar@192 backend % clear
(venv310) ajaykumar@192 backend % python - <<'PY'
import joblib, os
BASE = os.path.expanduser("~/Desktop/ShieldPatch/backend/models")
try:
    joblib.load(os.path.join(BASE, "best_model.joblib"))
    print("Model exists.")
except Exception as e:
    print("MODEL NOT TRAINED or CORRUPT:", e)
PY
MODEL NOT TRAINED or CORRUPT:
```

Fig. 8.1.1: Model Prediction Before Training

This chart illustrates how inconsistent and unproven the machine learning model is with respect to producing valid risk scores prior to training. Without historical learning patterns to develop risk results, the machine learning model's predictions will continue to show inconsistency as it will not have been trained with structured data on vulnerable assets.

```
(venv310) ajaykumar@192 backend % python - <<'PY'
import pandas as pd, os, numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

p = os.path.expanduser("~/Desktop/ShieldPatch/backend/data/ml_dataset_cleaned_fast.csv")
df = pd.read_csv(p).dropna(subset=["description_text"]).head(300)

X = df["description_text"].astype(str).tolist()
y = pd.to_numeric(df["cvss_score"], errors='coerce').fillna(0).values

tfidf = TfidfVectorizer(max_features=5000)
Xv = tfidf.fit_transform(X)

model = Ridge(alpha=1.0)
model.fit(Xv, y)

yp = model.predict(Xv)

print("=== AFTER TRAINING ACCURACY ===")
print("MSE :", mean_squared_error(y, yp))
print("R2  :", r2_score(y, yp))
PY
=== AFTER TRAINING ACCURACY ===
MSE : 0.10282852130778575
R2  : 0.8122703466436462
```

Fig. 8.1.2: Model Prediction After Training

This visual shows how well our model has learned to predict risk through TF_IDF vectorization and a regression approach. We can see that our trained model produces stable and meaningful risk scores, indicating that it is learning from the data set we used for training. With training, we are able to increase prediction accuracy and consistency considerably.

```
(venv310) ajaykumar@192 backend % python - <<'PY'
from joblib import load
import os
BASE = os.path.expanduser("~/Desktop/ShieldPatch/backend/models")
tfidf = load(os.path.join(BASE,"tfidf.joblib"))
clf = load(os.path.join(BASE,"best_model_plain.joblib"))

text = "Remote code execution vulnerability in Apache Struts allows attackers full system compromise."
Xv = tfidf.transform([text])
print("HIGH severity predicted:", clf.predict(Xv)[0])
PY
HIGH severity predicted: 7.301999999999998
```

Fig. 8.1.3: High Risk Vulnerability Prediction

The model's response to a critical vulnerability is illustrated in this figure. The model predicts that there is a high-risk rating for this vulnerability, indicating that it has the potential for exploitation and security impact. Therefore, it can be concluded that the model is effective at identifying critical vulnerabilities.

```
(venv310) ajaykumar@192 backend % python - <<'PY'
from joblib import load
import os
BASE = os.path.expanduser("~/Desktop/ShieldPatch/backend/models")
tfidf = load(os.path.join(BASE,"tfidf.joblib"))
clf = load(os.path.join(BASE,"best_model_plain.joblib"))

text = "Minor configuration issue that allows UI misalignment but no security impact."
Xv = tfidf.transform([text])
print("LOW severity predicted:", clf.predict(Xv)[0])
PY
LOW severity predicted: 6.472999999999997
```

Fig. 8.1.4: Medium / Low Risk Vulnerability Prediction

This figure displays the predicted outputs for a low/medium severity description. The resulting risk rating shows a lower value based on the security impact of this description being less severe. This indicates that the model can differentiate between different levels of vulnerabilities based on severity.

8.2 Functional Testing

Functionality Testing

FIELD	DETAILS
TEST ID	ML01
TEST NAME	Linear Regression – Risk Score Prediction
TEST DESCRIPTION	Validate prediction behavior of baseline Linear Regression model.

STEP	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	Send POST /predict_text	"buffer overflow"	Model returns numeric score (approx.)	JSON: { predicted: ~6.5 }	Pass
2	Safe text — low severity	"minor ui bug"	Predicted score < 4	JSON: { predicted: 3.1 }	Pass
3	Empty input	{ }	Error: "description_text required"	JSON: {error:"description_text required"}	Pass

Fig. 8.2.1: ML Risk Prediction Test Case (ML01)

Functionality Testing

FIELD	DETAILS
TEST ID	ML02
TEST NAME	RandomForest Risk Prediction
TEST DESCRIPTION	Verify that the trained RandomForest model generates stable scores.

STEP	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	POST /predict_text	"sql injection vulnerability"	Score approx 6–8 (High risk)	{ predicted: 7.0 }	Pass
2	POST /predict_text	"heap overflow in parser"	Score approx 6.0–7.5	{ predicted: 6.5 }	Pass
3	Invalid JSON	""	Error: invalid input	{error:"invalid JSON body"}	Pass
4	Batch request	[{"description_text":"sql"}, {"description_text":"xss"}]	Returns array of predictions	{count:2, predictions:[...]}	Pass

Fig. 8.2.2: Random Forest Risk Prediction Test Case (ML02)

FIELD	DETAILS
TEST ID	ML03
TEST NAME	XGBoost Risk Prediction
TEST DESCRIPTION	Validate predictive behavior of XGBoost model trained on TF-IDF features.

STEP	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	POST /predict_text	"RCE vulnerability in daemon"	Score approx 7–9	{ predicted: 7.4 }	Pass
2	POST /predict_text	"information disclosure only"	Score approx < 6	{ predicted: 5.2 }	Pass
3	Empty text	""	Error: missing text	{error:"description_text required"}	

Fig. 8.2.3: XGBoost Risk Prediction Test Case (ML03)

9. CONCLUSION

The ShieldPatch system has developed an automated/intelligent method of detecting vulnerabilities and managing patches. This project meets one of the major challenges facing the industry – how to prioritize which vulnerabilities have the biggest impact on cybersecurity and ensure that all exploits are patched promptly.

The ShieldPatch system applies the use of threat intelligence (CVE, EPSS, and ExploitDB) and uses a machine learning-based (ML) model to predict risks and rank order vulnerabilities. By using a TF-IDF vectorization method in combination with a Random Forest ML algorithm, the ShieldPatch system is able to identify and differentiate between high and low risks, assisting with better-informed patching decisions.

In addition to scanning for vulnerabilities that exist across systems, ShieldPatch scans for vulnerabilities existing in all existing files. The ShieldPatch also contains extensive files and file types that have been identified as containing potential vulnerabilities; the number of such files is continuously growing. However, the most significant aspect of ShieldPatch is that it also allows users to obtain patched versions of those files, which are then tested within a sandboxed environment utilizing Docker before being approved for deployment.

Accordingly, should one of the many patches fail, a rollback mechanism is available to maintain stability to the system. In addition to the abovementioned improvements to the usability of the ShieldPatch through the use of the dashboard and AI-powered chatbot, the overall system has demonstrated that it can effectively identify vulnerabilities and sufficiently provide the means to provide patch recommendations. Many of these same organizations have highlighted a need for an efficient means of implementing a reliable cybersecurity solution.

As such, the ShieldPatch supports both machine learning and automation-based efforts to develop practical and sustainable cybersecurity solutions, as well as improving decision-making and lowering manual effort associated with implementing effective cybersecurity solutions for an organization.

10. FUTURE ENHANCEMENTS

While ShieldPatch offers a viable approach to detecting vulnerabilities and suggesting patches; There is room for improvements to the functionality and scalability of the application.

One significant improvement would be the addition of fully automated patch deployment, which would allow the application to automatically install patches as they are detected; there would be no human intervention needed to install the patches. This would decrease the amount of time it takes to respond to vulnerabilities and improve security.

The current machine learning model that predicts the risk of vulnerability is mostly based on textual descriptions of potential vulnerabilities. Future enhancement should add additional context to the model by incorporating additional information such as the importance of the system, the network topology, the criticality of the asset, etc. There are also possibilities to develop more advanced machine learning algorithms, such as deep learning and graph-based algorithms that could help with the prediction of exploitation.

A key benefit to upgrading and improving the overall capabilities of ShieldPatch comes in the addition of cloud and enterprise-level environment support. The ability for ShieldPatch to run across both distributed environments and large system infrastructures will allow the full system to function efficiently.

Allowing for further improvement in the sandbox by adding support for cloud-based testing will also increase the speed and also improve the test-runs for each patch.

Greater capabilities in Natural Language Processing embedded in ShieldPatch will enhance the quality of responses that the AI provides, both in terms of accuracy and context awareness.

Integration of real-time Intrusion Detection and Prevention Systems (IDPS) into the overall security framework of the ShieldPatch product will also strengthen the ShieldPatch product overall security.

The ability to receive updates from threat intelligence sources and to periodically retrain the Machine Learning model will keep ShieldPatch current with the latest evolving threats against networks.

Overall, these improvements will provide ShieldPatch with greater power, scalability, and an overall level of automation that is unparalleled within the cybersecurity ecosystem.

BIBLIOGRAPHY

- [1] N. S. Harzevili et al., “A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning,” *ACM Computing Surveys*, vol. 57, no. 3, art. 55, pp. 1–36, Nov. 2024, doi: 10.1145/3699711.
- [2] M. S. Hoque et al., “An Improved Vulnerability Exploitation Prediction Model with Novel Cost Function and Custom Trained Word Vector Embedding,” *Sensors*, vol. 21, no. 4220, pp. 1–17, Jun. 2021, doi: 10.3390/s21124220.
- [3] J. Jacobs et al., “Exploit Prediction Scoring System (EPSS),” *Digital Threats: Research and Practice*, vol. 2, no. 3, art. 20, pp. 1–17, Jun. 2021, doi: 10.1145/3436242.
- [4] J. Jacobs et al., “Improving Vulnerability Remediation through Better Exploit-Aware Prioritization,” *Journal of Cybersecurity*, vol. 6, no. 1, pp. 1–12, 2020, doi: 10.1093/cybsec/tyaa015.
- [5] S. Kang and S. Yoo, “Language Models Can Prioritize Patches for Practical Program Patching,” in *Proc. Int. Workshop on Automated Program Repair (APR '22)*, Pittsburgh, USA, May 2022, pp. 1–8, doi: 10.1145/3524459.3527343.
- [6] S. Kirubakaran et al., “ILLATION: Improving Vulnerability Risk Prioritization by Learning Contextual Factors,” in *Proc. ICECAA 2023*, Coimbatore, India, pp. 1209–1213, doi: 10.1109/ICECAA58104.2023.10212345.
- [7] M. Esposito and D. Falessi, “VALIDATE: A Deep Dive into Vulnerability Prediction Datasets,” *Information and Software Technology*, vol. 170, art. 107448, 2024, doi: 10.1016/j.infsof.2024.107448.
- [8] V. Vasudevan and S. Narayanan, “Cyber Threat Intelligence Mining for Proactive Cybersecurity Defense,” *IEEE*, 2023, doi: 10.1109/IEEECON56657.2023.10126753.
- [9] M. Shanmugam et al., “SmartPatch: A Patch Prioritization Framework for Critical Systems,” *International Journal of Information Security Science*, vol. 13, no. 2, pp. 89–102, 2023.
- [10] Y. Zhang et al., “Exploitability Prediction of Vulnerabilities Based on Heterogeneous Graphs,” *Computers & Security*, vol. 138, art. 103560, 2024, doi: 10.1016/j.cose.2024.103560.